

```

class BullyAlgorithm:
    def __init__(self, processes, coordinator=None):
        self.processes = processes # list of process IDs
        self.coordinator = coordinator or max(processes)

    def is_alive(self, process_id):
        # Simulate which processes are alive; you can customize this
        return process_id in self.processes

    def hold_election(self, initiator):
        print(f"Process {initiator} starts election.")
        higher_processes = [p for p in self.processes if p > initiator]

        if not higher_processes:
            self.coordinator = initiator
            print(f"Process {initiator} becomes the new
coordinator.")
        else:
            for p in higher_processes:
                print(f"Process {initiator} sends election
message to {p}.")
            responses = [p for p in higher_processes if
self.is_alive(p)]

            if responses:
                print(f"Process {initiator} gets responses
from: {responses}")
                # One of the higher processes will take over
election
                highest = max(responses)
                self.hold_election(highest)
            else:
                self.coordinator = initiator
                print(f"No higher processes responded.
{initiator} becomes coordinator.")

# Example usage
processes = [1, 2, 3, 5, 6]
bully = BullyAlgorithm(processes)
bully.hold_election(initiator=3)

```

```

class RingAlgorithm:
    def __init__(self, processes):
        self.processes = sorted(processes)
        self.ring = self.processes[:]
        self.coordinator = None

```

```

def is_alive(self, process_id):
    return process_id in self.ring

def hold_election(self, initiator):
    print(f"Process {initiator} starts election.")
    election_list = [initiator]
    current_index = self.ring.index(initiator)

    while True:
        current_index = (current_index + 1) % len(self.ring)
        next_process = self.ring[current_index]

        if next_process == initiator:
            break

        print(f"Election message from {election_list[-1]} to {next_process}")
        election_list.append(next_process)

    winner = max(election_list)
    self.coordinator = winner
    print(f"Process {winner} is elected as the new coordinator.")

# Example usage
processes = [1, 2, 4, 6]
ring = RingAlgorithm(processes)
ring.hold_election(initiator=2)

```
